I am calculating mathematical problem where I need to compute the rotation angle between two vectors.
Context:
My current mathematical approach is:vector1Angle = arctan((B1y - A1y) / (B1x - A1x))vector2Angle = arctan((B2y - A2y) / (B2x - A2x))rotationAngle = vector2Angle - vector1Angle

Action:
Please explain if I should use:- np.arctan(y/x)or- np.arctan2(y, x)for this. I saw both in the documentation https://numpy.org/doc/stable/reference/generated/numpy.arctan.html#numpy.arctan
Especially the "See also" section where arctan2 is mentioned. Please explain me
what is the difference between arctan and arctan2.

Show less

Marshaled knowledge to clarify arctan versus arctan2 distinctions >

`arctan` vs `arctan2` — use `arctan2`

For your use case, **always use `np.arctan2(y, x)`**. Here's why:

The core problem with `arctan(y/x)`

`arctan` only returns angles in **(-90°, +90°)** — a half-circle. It cannot distinguish which quadrant your vector is actually in, because dividing `y/x` loses the sign information of each component individually.

Vector direction	`y/x` ratio	`arctan(y/x)`	`arctan2(y, x)`
Northeast (+x, +y)	+1	**+45°** ✓	**+45°** ✓
Southwest (-x, -y)	+1	**+45°** ✗	**-135°** ✓
Northwest (-x, +y)	-1	**-45°** ✗	**+135°** ✓
Southeast (+x, -y)	-1	**-45°** ✓	**-45°** ✓

Southwest and Northeast produce the **same ratio**, so `arctan` maps them to the same angle — that's a silent, hard-to-debug error.

What `arctan2` does differently

`arctan2(y, x)` takes **both components separately**, preserving their individual signs, and returns the full **(-180°, +180°]** range. It also handles the `x=0` edge case (vertical vectors) without a division-by-zero crash.

Your corrected code

```python
import numpy as np

# Vector 1: from A1 to B1
vector1_angle = np.arctan2(B1y - A1y, B1x - A1x)  # note: y first, then x

# Vector 2: from A2 to B2
vector2_angle = np.arctan2(B2y - A2y, B2x - A2x)

rotation_angle = vector2_angle - vector1_angle
```

One extra tip — normalize the result

The subtraction can push the result outside (-180°, +180°], e.g. if vector1 is at 170° and vector2 is at -170°, you'd get -340° instead of the correct +20°. Wrap it:

```python
rotation_angle = (vector2_angle - vector1_angle + np.pi) % (2 * np.pi) - np.
```

This guarantees your rotation angle always stays in the **(-180°, +180°]** range, which is almost always what you want.